

Monte Carlo Tree Search Implementation for Tic Tac Toe

Name: Abhishek Patil

NUID: 002319098

Name: Shivani Sugurushetty

NUID: 002305622

GitHub Link: <https://github.com/Abhipatil1008/DSAIPG/tree/tictactoe>,
<https://github.com/Abhipatil1008/DSAIPG/tree/project-main>,
<https://github.com/shivanisugurushetty/DSAIPG>

Demo Recording: https://northeastern-my.sharepoint.com/:v/g/person/patil_abhishek_northeastern_edu/EdmWSMAazlNFu0Ntmk2PLCgB2HHOUrkZI9Ma0UfKCTvd3w?nav=eyJyZWZlcnJhbEluZm8iOmsicmVmZXJyYWxBcHAIa0iJPbmVEcmll2ZUZvckJlc2luZXNzIiwicmVmZXJyYWxBcHBhbGF0Zm9ybSI6IldlYiIsInJlZmVycm

Note: Full implementation, benchmarking and test results are in the **project-main** branch. For code changes and commit history check the above Github links.

Overview:

Tic Tac Toe is implemented by Monte Carlo Tree Search (MCTS). The aim was to design an AI capable of making the best moves using MCTS and benchmark the performance across various simulation levels. Code is developed in Java and follows an object-oriented design pattern with unit testing and benchmarking. Tic Tac Toe is a two-player game played on a 3*3 board. Each player takes turns placing their mark (X or O) until one player aligns three marks in a row, column, or diagonal or the board is filled.

Implementation:

TicTacToe.java – Main Game Logic. Run to play the AI game vs human.

Position.java – Representation of Board.

TicTacToeNode.java – MCTS Tree Node.

MCTS.java – Search Algorithm.

- Nodes are selected using Upper Confidence Bound (UCT)
- For every unexplored move, a new node is added.
- Games are simulated to the end with either random or priority-based move selection.
- Results are circulated back to the root.
- Heuristic enhancements to block against human win.

TicTacToeTest.java

PositionTest.java

TicTacToeNodeTest.java

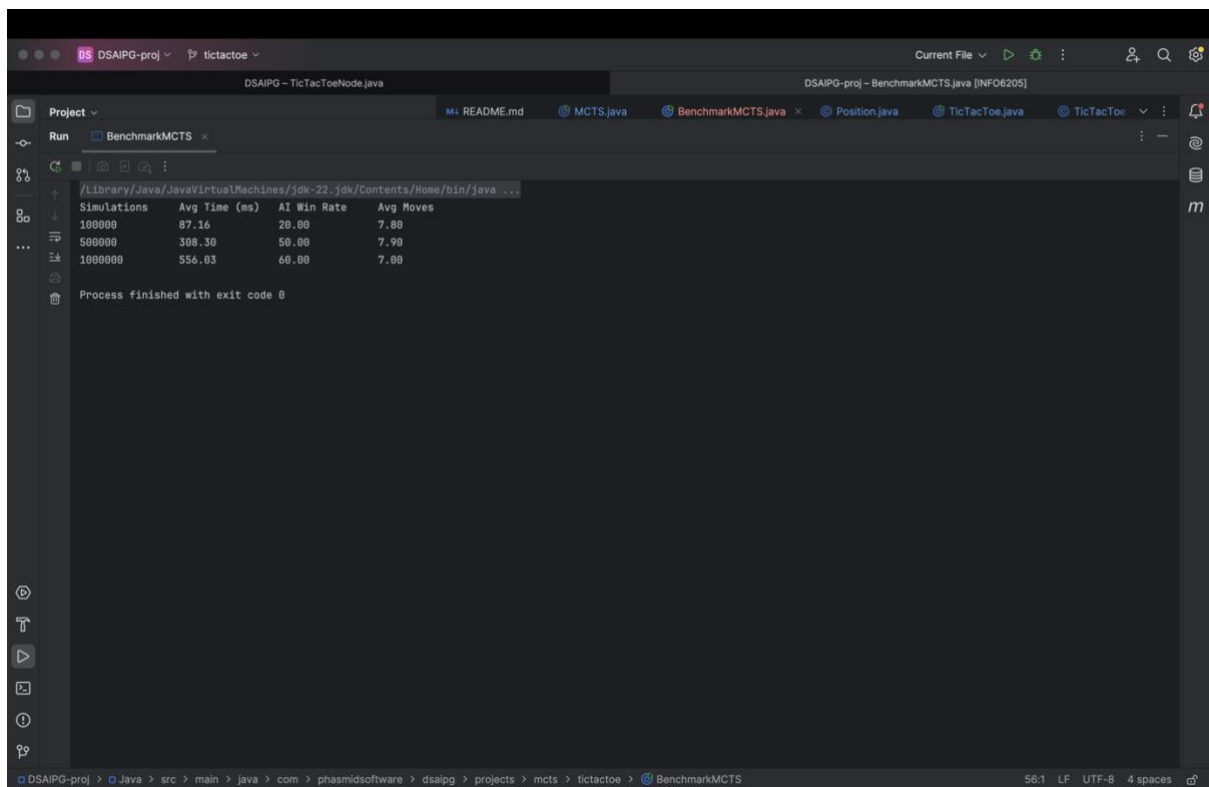
MCTSTest.java

BenchmarkMCTS.java – performance data at different simulation depths.

Benchmarking and Performance Analysis:

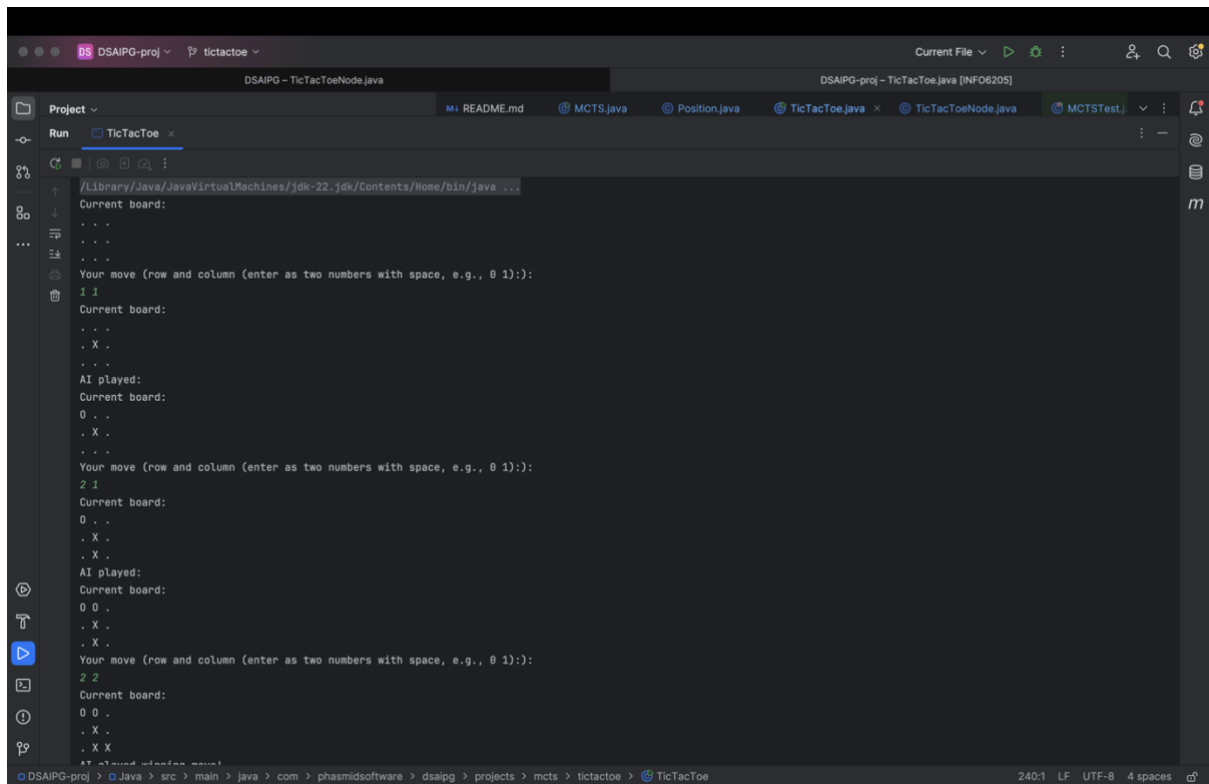
Simulations	Avg Time (ms)	AI Win Rate (%)	Avg Moves
100000	87.16	20.00	7.80
500000	308.30	50.00	7.90
1000000	556.03	60.00	7.00

Higher simulations improve win rate and decision quality but increase computation time.



```
DS AIPG-proj  tictactoe  Current File  [INFO6205]
DSAIPG - TicTacToeNode.java  DSAIPG-proj - BenchmarkMCTS.java [INFO6205]
Project  README.md  MCTS.java  BenchmarkMCTS.java  Position.java  TicTacToe.java  TicTacToe
Run  BenchmarkMCTS
/Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java ...
Simulations  Avg Time (ms)  AI Win Rate  Avg Moves
100000  87.16  20.00  7.80
500000  308.30  50.00  7.90
1000000  556.03  60.00  7.00
Process finished with exit code 0
DSAIPG-proj  Java  src  main  java  com  phasmidsoftware  dsaipg  projects  mcts  tictactoe  BenchmarkMCTS  56:1  LF  UTF-8  4 spaces
```

Output screenshots and test cases:



```
DSAIPG-proj - TicTacToeNode.java
DSAIPG-proj - TicTacToe.java [INFO6205]

Project - TicTacToe x
Run - TicTacToe x

/Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java ...
Current board:
. . .
. . .
. . .

Your move (row and column (enter as two numbers with space, e.g., 0 1):):
1 1
Current board:
. . .
. X .
. . .

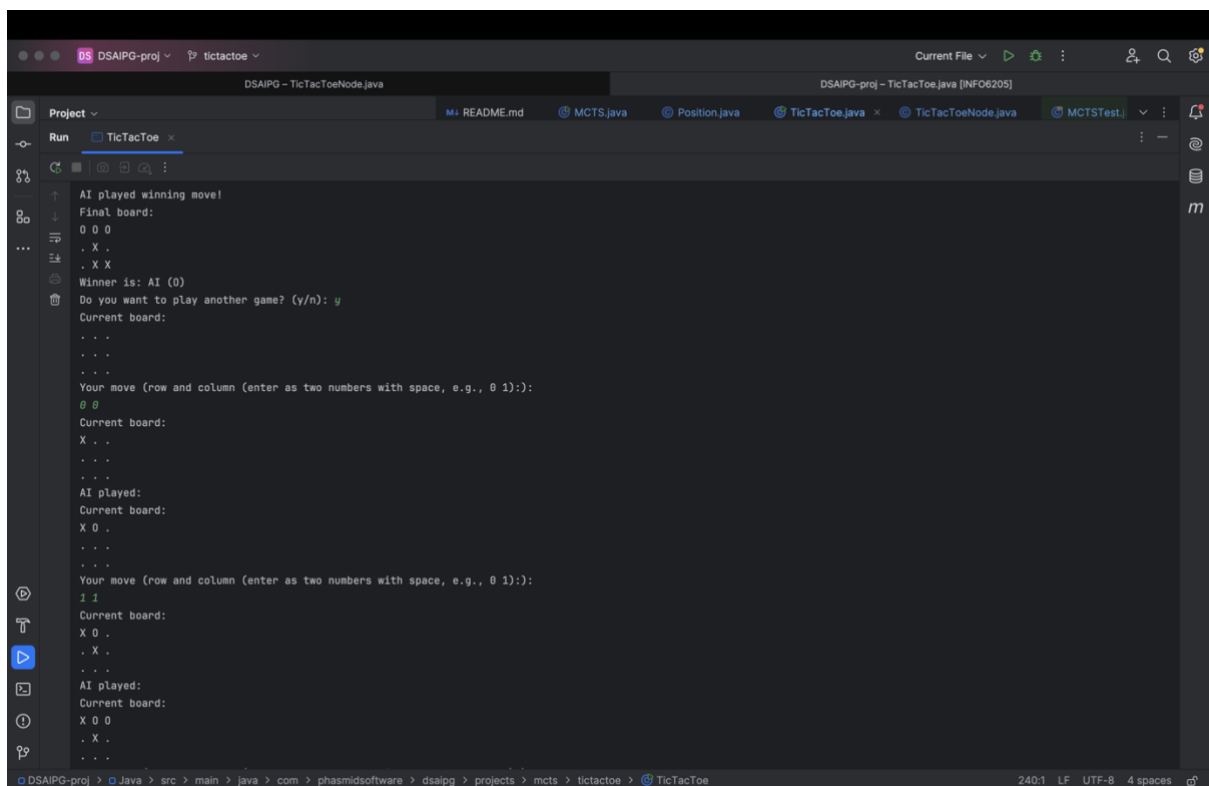
AI played:
Current board:
0 . .
. X .
. . .

Your move (row and column (enter as two numbers with space, e.g., 0 1):):
2 1
Current board:
0 . .
. X .
. X .

AI played:
Current board:
0 0 .
. X .
. X .

Your move (row and column (enter as two numbers with space, e.g., 0 1):):
2 2
Current board:
0 0 .
. X .
. X X

AT -land window posit
```



```
DSAIPG-proj - TicTacToeNode.java
DSAIPG-proj - TicTacToe.java [INFO6205]

Project - TicTacToe x
Run - TicTacToe x

AI played winning move!
Final board:
0 0 0
. X .
. X X

Winner is: AI (0)
Do you want to play another game? (y/n): y
Current board:
. . .
. . .
. . .

Your move (row and column (enter as two numbers with space, e.g., 0 1):):
0 0
Current board:
X . .
. . .
. . .

AI played:
Current board:
X 0 .
. . .
. . .

Your move (row and column (enter as two numbers with space, e.g., 0 1):):
1 1
Current board:
X 0 .
. X .
. . .

AI played:
Current board:
X 0 0
. X .
. . .
```

```
DSAIPG-proj  tictactoe  Current File  [INFO6205]
DSAIPG - TicTacToeNode.java  DSAIPG-proj - TicTacToe.java [INFO6205]
Project  README.md  MCTS.java  Position.java  TicTacToe.java  TicTacToeNode.java  MCTSTest
Run  TicTacToe
Current board:
X 0 0
. X .
. . .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
2 2
Final board:
X 0 0
. X .
. . X
Winner is: You (X)
Do you want to play another game? (y/n): y
Current board:
. . .
. . .
. . .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
2 1
Current board:
. . .
. . .
. X .
AI played:
Current board:
0 . .
. . .
. X .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
0 1
Current board:
0 X .
. . .
. X .
AI played:
AT end-
DSAIPG-proj > Java > src > main > java > com > phasmidsoftware > dsaipg > projects > mcts > tictactoe > TicTacToe 240:1 LF UTF-8 4 spaces
```

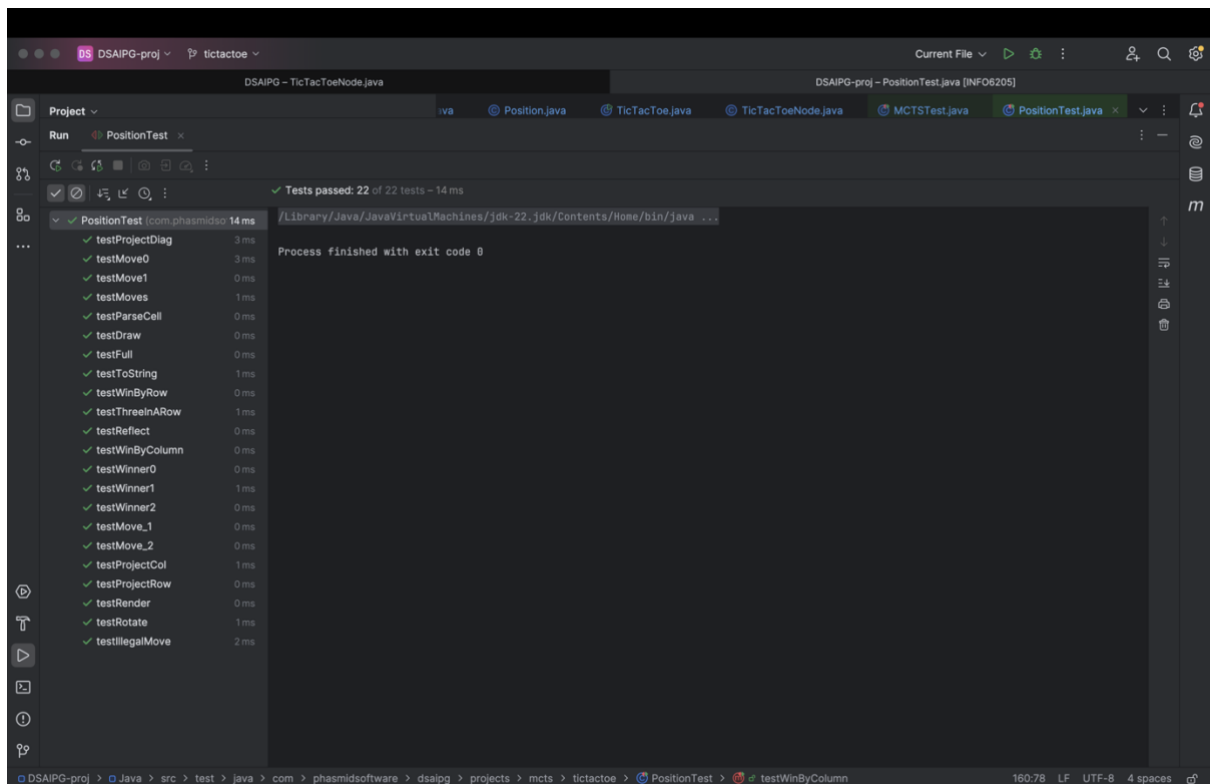
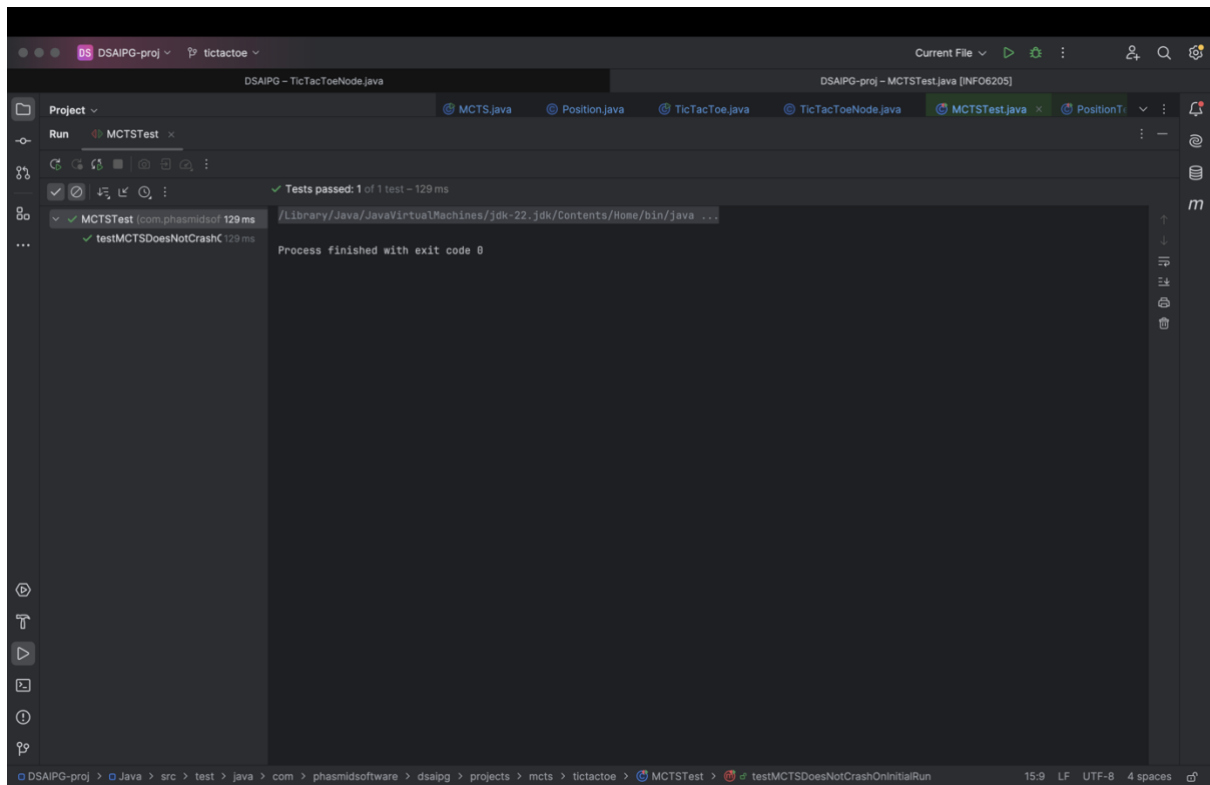
```
DSAIPG-proj  tictactoe  Current File  [INFO6205]
DSAIPG - TicTacToeNode.java  DSAIPG-proj - TicTacToe.java [INFO6205]
Project  README.md  MCTS.java  Position.java  TicTacToe.java  TicTacToeNode.java  MCTSTest
Run  TicTacToe
AI played:
Current board:
0 X 0
. . .
. X .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
1 1
Final board:
0 X 0
. X .
. X .
Winner is: You (X)
Do you want to play another game? (y/n): y
Current board:
. . .
. . .
. . .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
1 2
Current board:
. . .
. . X
. . .
AI played:
Current board:
0 . .
. . X
. . .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
0 1
Current board:
0 X .
. . X
. . .
DSAIPG-proj > Java > src > main > java > com > phasmidsoftware > dsaipg > projects > mcts > tictactoe > TicTacToe 240:1 LF UTF-8 4 spaces
```

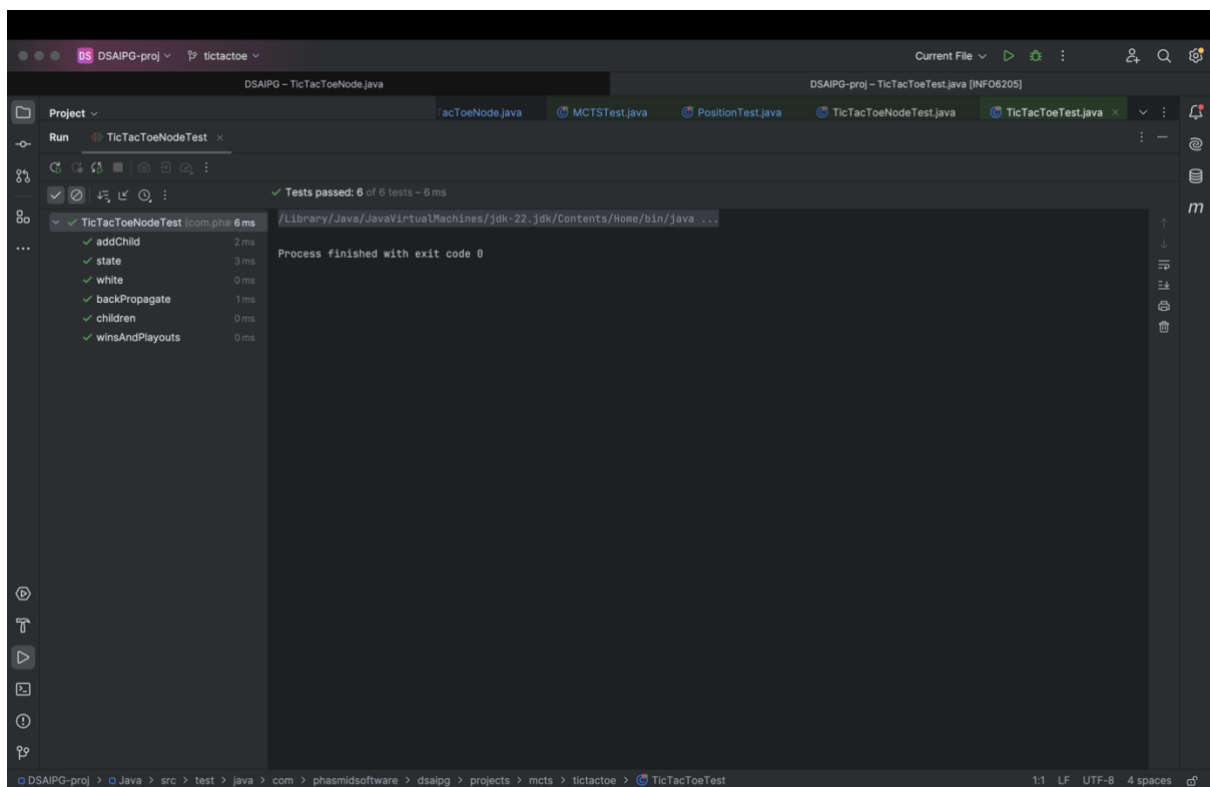
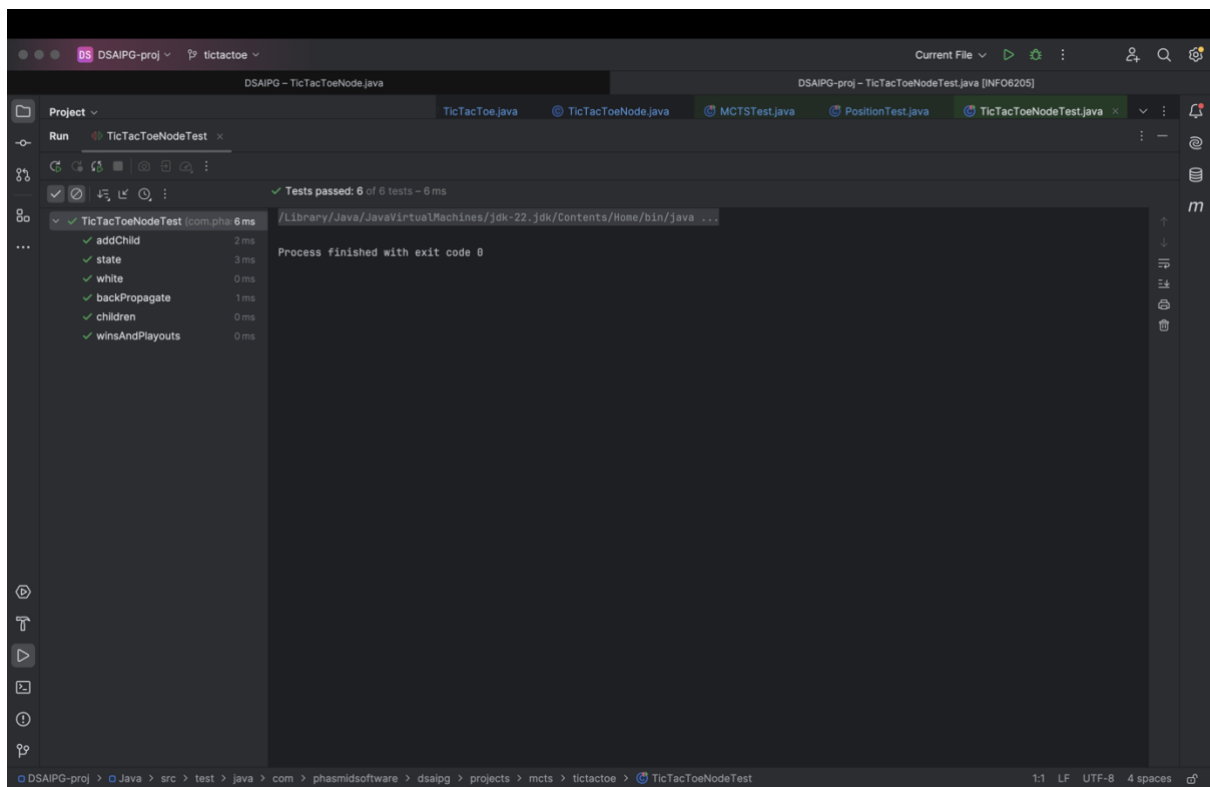
```
DSAIPG-proj  tictactoe  Current File  DSAIPG-proj - TicTacToeNode.java  DSAIPG-proj - TicTacToe.java [INFO6205]
Project  README.md  MCTS.java  Position.java  TicTacToe.java  TicTacToeNode.java  MCTSTest
Run  TicTacToe
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
0 1
Current board:
0 X .
. . X
. . .
AI played:
Current board:
0 X 0
. . X
. . .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
1 1
Current board:
0 X 0
. X X
. . .
AI played:
Current board:
0 X 0
0 X X
. . .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
2 0
Current board:
0 X 0
0 X X
X . .
AI played:
Current board:
0 X 0
0 X X
X 0 .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
DSAIPG-proj  Java  src  main  java  com  phasmidsoftware  dsaipg  projects  mcts  tictactoe  TicTacToe  240:1  LF  UTF-8  4 spaces
```

```
DSAIPG-proj  tictactoe  Current File  DSAIPG-proj - TicTacToeNode.java  DSAIPG-proj - TicTacToe.java [INFO6205]
Project  README.md  MCTS.java  Position.java  TicTacToe.java  TicTacToeNode.java  MCTSTest
Run  TicTacToe
Current board:
0 X 0
0 X X
X 0 .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
2 2
Final board:
0 X 0
0 X X
X 0 X
It's a draw!
Do you want to play another game? (y/n): y
Current board:
. . .
. . .
. . .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
1 1
Current board:
. . .
. X .
. . .
AI played:
Current board:
0 . .
. X .
. . .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
2 1
Current board:
0 . .
. X .
. X .
AI played:
DSAIPG-proj  Java  src  main  java  com  phasmidsoftware  dsaipg  projects  mcts  tictactoe  TicTacToe  240:1  LF  UTF-8  4 spaces
```

```
DSAIPG-proj  tictactoe  Current File  [INFO6205]
DSAIPG - TicTacToeNode.java  DSAIPG-proj - TicTacToe.java [INFO6205]
Project  TicTacToe
Run  TicTacToe
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
2 1
Current board:
0 . .
. X .
. X .
AI played:
Current board:
0 0 .
. X .
. X .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
0 2
Current board:
0 0 X
. X .
. X .
AI played:
Current board:
0 0 X
0 X .
. X .
Your move (row and column (enter as two numbers with space, e.g., 0 1):)
2 0
Final board:
0 0 X
0 X .
X X .
Winner is: You (X)
Do you want to play another game? (y/n):
Process finished with exit code 130 (interrupted by signal 2:SIGINT)
DSAIPG-proj  Java  src  main  java  com  phasmidsoftware  dsaipg  projects  mcts  tictactoe  TicTacToe  240:1  LF  UTF-8  4 spaces
```

```
DSAIPG-proj  tictactoe  Current File  [INFO6205]
DSAIPG - TicTacToeNode.java  DSAIPG-proj - MCTS.java [INFO6205]
Project  MCTS
Run  MCTS
/Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java ...
Best state found:
TicTacToe{
1,-1,-1
-1,-1,-1
-1,-1,-1
}
Process finished with exit code 0
DSAIPG-proj  Java  src  main  java  com  phasmidsoftware  dsaipg  projects  mcts  tictactoe  MCTS  simulate  91:44  LF  UTF-8  4 spaces
```





Conclusion:

MCTS algorithm proves effectiveness for the game of Tic-Tac-Toe. Through simulations and UCT-based selection, AI consistently plays the best and near-optimal moves. Benchmarking shows that a greater number of simulations improved decision quality. This implementation proves the correctness and efficiency of MCTS in fully observable, deterministic, two-player games with limited complexity.

References:

https://en.wikipedia.org/wiki/Monte_Carlo_tree_search
<https://arxiv.org/abs/2103.04931>
<https://int8.io/monte-carlo-tree-search-beginners-guide/>

Monte Carlo Tree Search Implementation for Priests and Devils

Overview:

The Priests and Devils is a well-known puzzle that involves moving three priests and three devils safely across a river using a boat. At no point should devils outnumber priests on either bank, if so, the priests will be eaten.

Implementation:

We implemented MCTS with the following steps:

- Selection – Traverse the tree using UCT (Upper Confidence Bound for trees).
- Expansion – Add valid child to the tree. If a goal state is found, prioritize and return.
- Simulation – Random playout from the expanded node to a terminal state.
- Backpropagation – Update the win/playout statistics. Goal states are given maximum reward.

Core Classes:

- Game state – Represents the state of riverbanks and boat position.
- MCTPriestDevilState – State interface for MCTS.
- PriestDevilNode – Node interface for MCTS.
- PriestDevilMCTS – MCTS logic with UCT-based selection.
- BenchmarkPriestDevilMCTS – Runs performance evaluations.

Test Classes: Valid state transitions, Goal state detection and Invalid move rejection.

- GameStateTest
- MoveGeneratorTest
- PriestDevilGameTest
- PriestDevilMCTSTest
- PriestDevilNodeTest

Benchmarking and performance analysis:

Simulations	Avg. Time (ms)	Success Rate (%)	Avg. Steps
1000	7.43	100.00	1.00
5000	6.77	100.00	1.00
10000	5.43	100.00	1.00

MCTS algorithm consistently solves the priests and devils puzzle with a 100% success rate in all tested configurations. The average computation time decreases with higher simulation counts, indicating improves efficiency and consistent goal recognition.

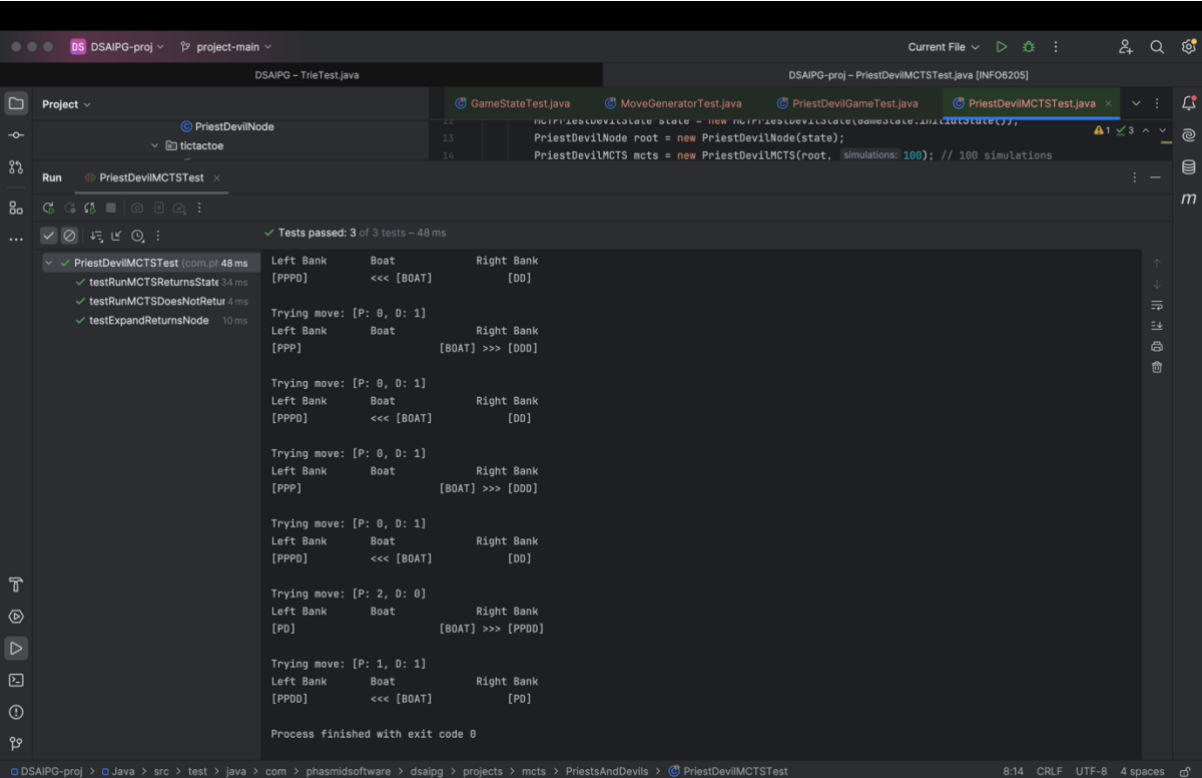
```
DSAIPG-proj - project-main
DSAIPG - TrieTest.java
DSAIPG-proj - BenchmarkPriestDevilMCTS.java [INFO:205]
Project
  > idea
  > apJava
Run
  BenchmarkPriestDevilMCTS
  [0] [BOAT] >>> [PPPD]
  Trying move: [P: 0, D: 1]
  Left Bank Boat Right Bank
  [00] <<< [BOAT] [PPPD]
  Trying move: [P: 0, D: 1]
  Left Bank Boat Right Bank
  [D] [BOAT] >>> [PPPD]
  Trying move: [P: 1, D: 0]
  Left Bank Boat Right Bank
  [PD] <<< [BOAT] [PPPD]
  Goal condition met!
  Goal condition met!
  Trying move: [P: 1, D: 1]
  Goal condition met!
  Left Bank Boat Right Bank
  [] [BOAT] >>> [PPPD]
  Goal condition met!
  Goal condition met!
  Simulations Avg Time (ms) Success Rate (%) Avg Steps
  10000 5.43 100.00 1.00
  1000 7.43 100.00 1.00
  5000 6.77 100.00 1.00
  10000 5.43 100.00 1.00
  Process finished with exit code 0
```

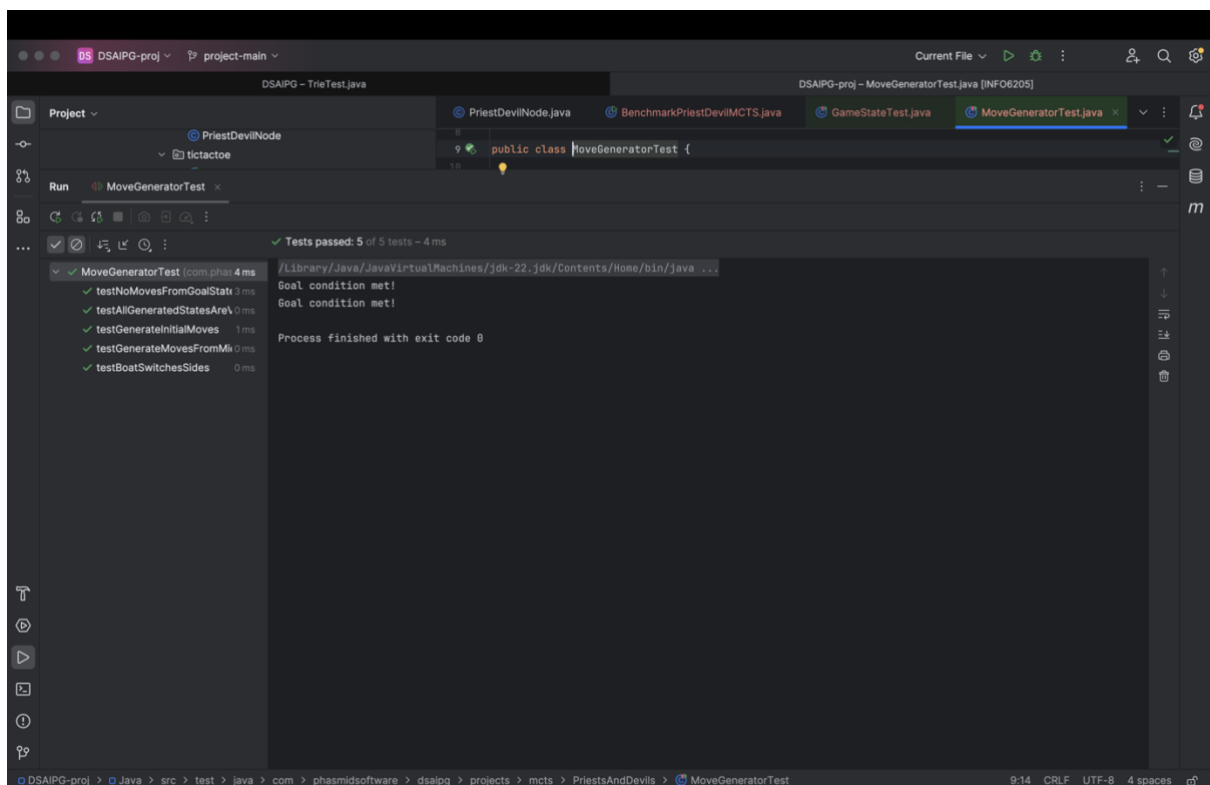
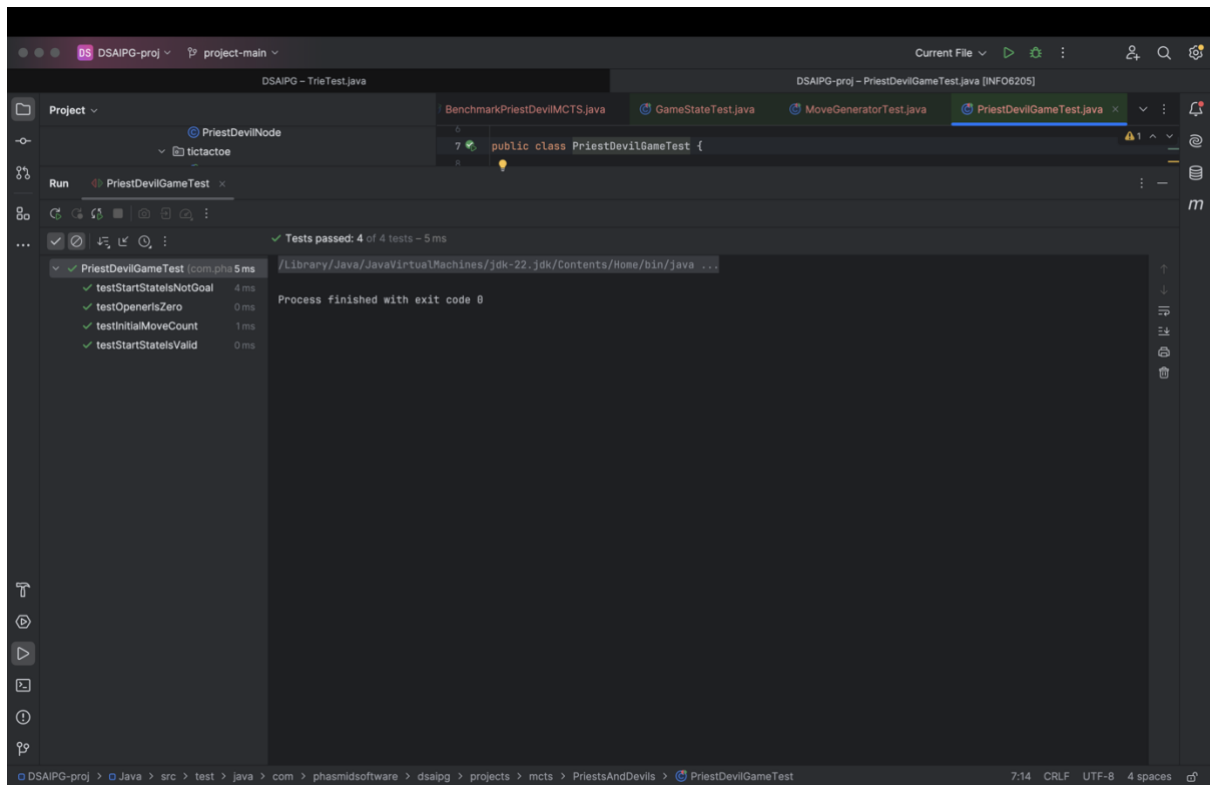
Output screenshots and test cases:

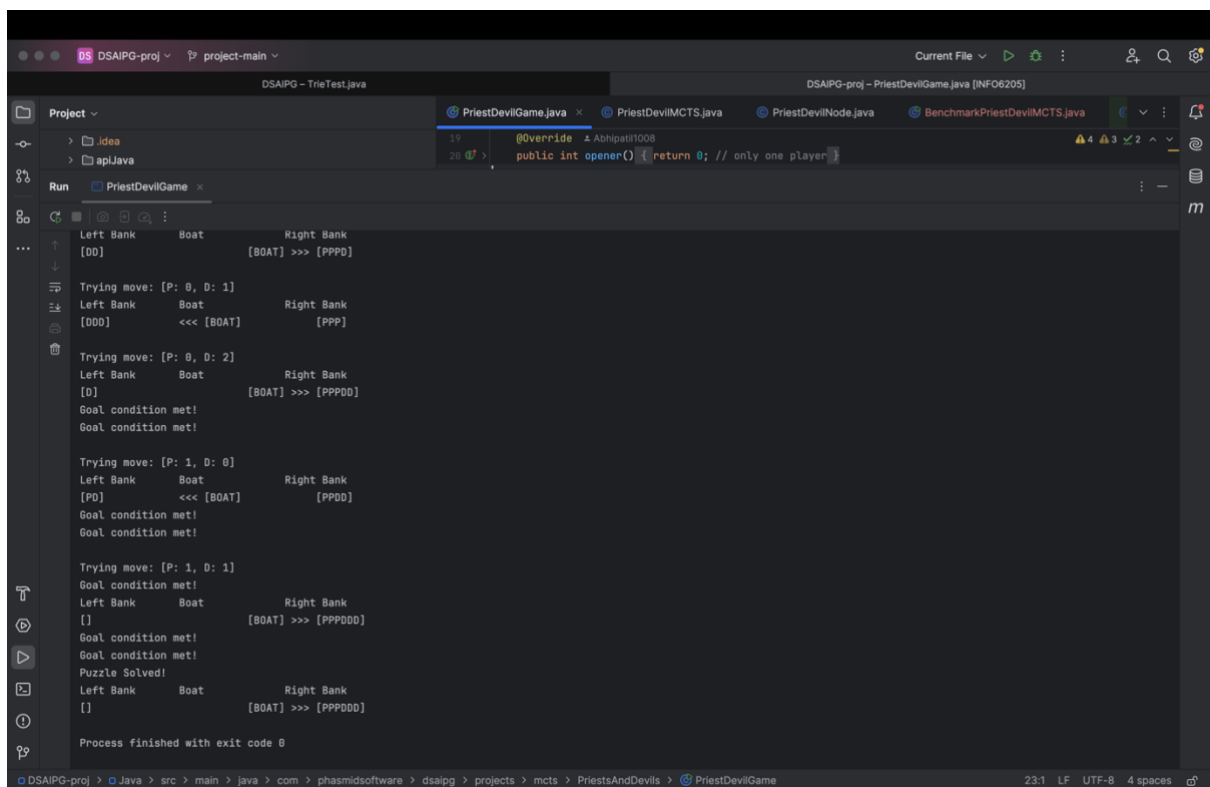
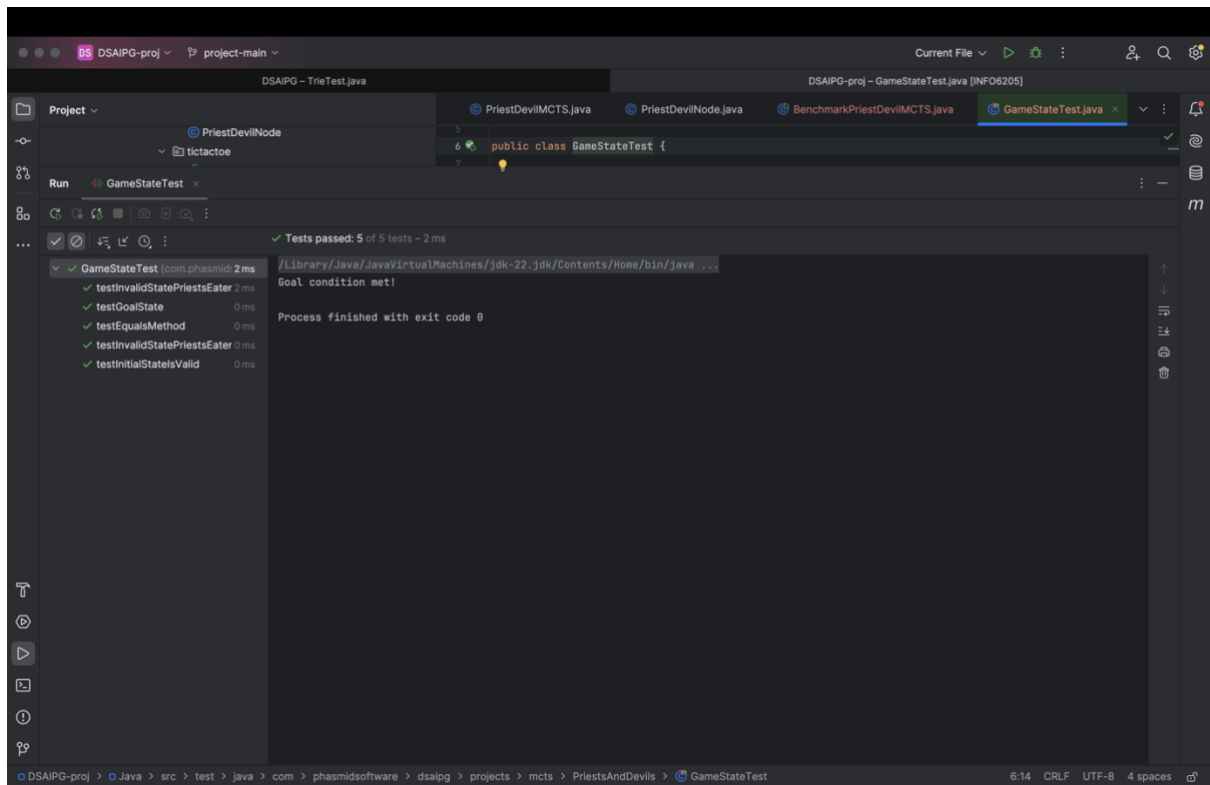
```
DSAIPG - tictactoe
PlayGame
Run
  PlayGame
  "C:\Program Files\Java\jdk-19\bin\java.exe" ...
  Welcome to Priests and Devils - Human Mode
  Left Bank Boat Right Bank
  [PPPD] <<< [BOAT] []
  Enter number of Priests to move (0-2):
  0
  Enter number of Devils to move (0-2):
  2
  Left Bank Boat Right Bank
  [PPPD] [BOAT] >>> [DD]
  Enter number of Priests to move (0-2):
  0
  Enter number of Devils to move (0-2):
  1
  Left Bank Boat Right Bank
  [PPPD] <<< [BOAT] [D]
  Enter number of Priests to move (0-2):
  0
  Enter number of Devils to move (0-2):
  2
  Left Bank Boat Right Bank
  [PPP] [BOAT] >>> [DDD]
  Enter number of Priests to move (0-2):
  0
  Enter number of Devils to move (0-2):
  1
  Left Bank Boat Right Bank
  [PPPD] <<< [BOAT] [DD]
  Enter number of Priests to move (0-2):
```

```
Project > Run > PlayGame > [PPPD] <<< [BOAT] [DD]
Enter number of Priests to move (0-2):
2
Enter number of Devils to move (0-2):
0
Left Bank      Boat      Right Bank
[PD]           [BOAT] >>> [PPDD]
Enter number of Priests to move (0-2):
1
Enter number of Devils to move (0-2):
1
Left Bank      Boat      Right Bank
[PPDD] <<< [BOAT] [PD]
Enter number of Priests to move (0-2):
2
Enter number of Devils to move (0-2):
0
Left Bank      Boat      Right Bank
[DD]           [BOAT] >>> [PPPD]
Enter number of Priests to move (0-2):
0
Enter number of Devils to move (0-2):
1
Left Bank      Boat      Right Bank
[DDD] <<< [BOAT] [PPP]
Enter number of Priests to move (0-2):
0
Enter number of Devils to move (0-2):
2
```

```
Project > Run > PlayGame > Enter number of Priests to move (0-2):
0
Enter number of Devils to move (0-2):
1
Left Bank      Boat      Right Bank
[DDD] <<< [BOAT] [PPP]
Enter number of Priests to move (0-2):
0
Enter number of Devils to move (0-2):
2
Left Bank      Boat      Right Bank
[DD]           [BOAT] >>> [PPPPDD]
Enter number of Priests to move (0-2):
0
Enter number of Devils to move (0-2):
1
Left Bank      Boat      Right Bank
[DD] <<< [BOAT] [PPPD]
Enter number of Priests to move (0-2):
0
Enter number of Devils to move (0-2):
2
Left Bank      Boat      Right Bank
[] [BOAT] >>> [PPPPDD]
Goal condition met!
Congratulations! You solved the puzzle!
Process finished with exit code 0
```







Conclusion:

MCTS implementation for this game successfully demonstrates the algorithm's ability to solve a sequential decision-making puzzle i.e. priests and devils. At 1000 simulations, the algorithm has achieved high success rates. Benchmarking results showed that the performance and success rate improved with more simulations, making MCTS a reliable strategy for a single-player problems.

References:

https://en.wikipedia.org/wiki/Monte_Carlo_tree_search
<https://arxiv.org/abs/2103.04931>
<https://int8.io/monte-carlo-tree-search-beginners-guide/>